

Paquetes, Módulos y Especificadores de Acceso

Paquetes y espacios de nombres

- Un **paquete en JAVA** es una colección de componentes de software (clases, interfaces, tipos enumerativos, anotaciones) con nombre. Los paquetes son útiles para **agrupar componentes de software** relacionados y definir un **espacio de nombres común** a las entidades contenidas en él.
- Las clases e interfaces esenciales (core) de la plataforma JAVA están ubicadas en un paquete cuyo nombre comienza con **java** y luego una serie de subnombres, resultando en un nombre jerárquico.
 - Las clases e interfaces fundamentales de JAVA están ubicadas en el paquete **java.lang**.
 - Las clases e interfaces utilitarias están en el paquete **java.util**.
 - Las clases e interfaces usadas para realizar I/O están en el paquete **java.io**.
 - Las clases e interfaces usadas para establecer conexiones con hosts remotos, que usan protocolos de comunicación de redes, etc. están en el paquete **java.net**.
 - Las clases e interfaces usadas para realizar funciones matemáticas y trigonométricas están en el paquete **java.math**.
 - Las clases e interfaces usadas para construir interfaces gráficas de usuario están en **java.awt**.

Paquetes, espacios de nombres y módulos

- Los paquetes a su vez pueden contener subpaquetes, como por ej. el paquete `java.awt` contiene los subpaquetes `java.awt.event` y `java.awt.image`; el paquete `java.lang` contiene los subpaquetes `java.lang.reflect` y `java.util.regex`.
- A partir de Java 9, los paquetes se agrupan en módulos con nombre. Los nombres de los módulos que forman parte del núcleo estándar de JSE comienzan con **java** y los que los de soporte comienzan con **jdk**. Por ejemplo el paquete `org.w3c.dom.html` que implementa estándares definidos por la W3C está incluido en el módulo **`jdk.xml.dom`** y el paquete `java.util` que contiene clases como `ArrayList` se encuentra en el módulo **`java.base`**.
- Todas las clases, interfaces, tipos enumerativos y anotaciones tienen un **nombre simple o no calificado** y un **nombre completo o calificado**. El **simple** es el que usamos para **definir la clase** y el **completo o calificado** es el que incluye como **prefijo el nombre del paquete** al que pertenece la clase. Por ej. la clase `String` es parte del paquete `java.lang`, su nombre simple es `String` y su nombre completo es `java.lang.String`.

Módulos

- Un **módulo** es una **agrupación de paquetes relacionados** bajo un **nombre único** para toda la aplicación.
- Un módulo declara **qué necesita** (*requires*) y **qué expone** (*exports*) al resto de la aplicación.
- Un **módulo** se declara en el archivo **module-info.java** (archivo descriptor del módulo).

```
module ar.com.laplataautos.gestion {  
    requires java.sql;           | paquetes que se necesitan  
    requires java.logging;      |  
    exports ar.com.laplataautos.modelo; | paquetes que se exportan  
    exports ar.com.laplataautos.servicios; |  
    // ar.com.laplataautos.interno NO se exporta  
}
```

← nombre del módulo

Nombres únicos de paquetes

Una de los propósitos más importantes de los paquetes es el de **dividir el espacio de nombres global de JAVA** y **evitar colisiones de nombres entre clases**. Por ej. el nombre del paquete permite diferenciar la **interface `java.util.List`** de la **clase `java.awt.List`**.

¿Cómo hacemos para que los nombres de los paquetes sean distintos?

Un esquema de nombres posible podría ser **usar el nombre invertido del dominio de Internet como prefijo de todos los nombres de paquetes**.

Por ej. si consideramos el nombre del dominio de Internet del LINTI, **`linti.unlp.edu.ar`**, los nombres de paquetes comenzarán con **`ar.edu.unlp.linti`**.

Luego con algún criterio se debe decidir cómo particionar el nombre después de **`ar.edu.unlp.linti`**.

Al ser único el nombre del dominio de Internet, ningún otro desarrollador u organización que respete esta regla definirá un paquete con el mismo nombre.

`ar.edu.unlp.linti.graficos`
`ar.gov.gba.ec.rrhh.sueldos`
`ar.com.afip.estadisticas.reportes`

Nombres únicos de paquetes y módulos

Pautas para elegir nombres únicos de paquetes:

- Si formamos equipos de desarrollo y las clases desarrolladas serán usadas por otros equipos y que se combinarán con múltiples clases desconocidas por nosotros/as, es importante que los nombres de los paquetes sean globalmente únicos. Por ej. desarrolladores de compañías de software, de comunidades de software libre, etc.
- Si estamos desarrollando una aplicación JAVA, cuyas clases no están concebidas para ser usadas por otros/as programadoras fuera de nuestro equipo de trabajo, podemos elegir un esquema de nombres de paquetes que se ajuste a nuestra conveniencia. En este caso conocemos el nombre completo del conjunto de clases que nuestra aplicación necesita para el *deployment* y no tendremos imprevistos por conflictos de nombres.

Esta misma distinción aplica al nombre de los módulos: un módulo público necesita un nombre globalmente único basado en dominios inversos para evitar conflictos. Un módulo interno de una aplicación propia puede darse el lujo de usar un nombre más simple.



La palabra clave *package*

En JAVA las clases e interfaces típicamente se agrupan en paquetes usando como primer *token* del archivo fuente la palabra clave **package** seguido por el nombre del paquete.

package ar.com.laplataautos;

```
public class Vehiculo {  
    private String marca;  
    private String nroMotor;  
    public String getMarca(){  
        return marca;  
    }  
    public String getNroMotor(){  
        return nroMotor;  
    }  
}
```

module-info.java del módulo ar.com.laplataautos

```
module ar.com.laplataautos {  
    exports ar.com.laplataautos;  
}
```

Para que otro módulo pueda usar **Vehiculo**, el módulo **ar.com.laplataautos** tiene que exportar ese paquete..

Si se omite la palabra clave **package** en la definición de una clase, la misma se ubicará en el paquete predeterminado conocido como **default package**. Este mecanismo resulta útil sólo para realizar pequeñas pruebas o testeos de algoritmos o lógica, sin embargo no es una buena práctica de programación.

Importar tipos de datos

El mecanismo predeterminado para incluir nombres de clases o interfaces existentes es usar el **nombre completo** de la clase o interface.

Por ej. si estamos escribiendo código que manipula un archivo vamos a necesitar usar la clase **File** que pertenece al paquete **java.io**, entonces deberíamos escribir **java.io.File** cada vez que usamos la clase.

¿Cuándo usar el **nombre simple** de un tipo de datos?

- Si usamos clases e interfaces del paquete **java.lang**. *Importación automática*
- Si usamos clases e interfaces que están definidas en el mismo paquete de la clase o interface que estamos escribiendo.
- Si incluimos tipos al espacio de nombres con la sentencia **import**.

Es posible explícitamente importar tipos de datos desde otros paquetes al espacio de nombres actual usando la sentencia **import**.

Importación de un tipo

// el módulo grafico exporta este paquete

import ar.edu.unlp.linti.graficos.Rectangulo;

import java.io.File;

import java.io.PrintWriter;

import ar.edu.unlp.linti.graficos.*;

import java.io.*;

Es posible usar el nombre simple de las clases por ej. Rectangulo, File, PrintWriter

Importación de tipos por demanda

(los tipos son importados a medida que se necesitan)

La sintaxis de importación de tipos por demanda no se aplica a subpaquetes.



Importar tipos de datos: con y sin módulos

Sin module-info.java (classpath / unnamed module)

- Se puede importar **cualquier** clase o interface **public** de cualquier JAR que esté en el classpath.
- No importa si la clase fue pensada como “detalle interno”: si es **public**, es importable.
- Es un modo muy común de uso.

```
// classpath: biblioteca-pdf.jar
import biblioteca.pdf.interno.Parser;
import biblioteca.pdf.api.Documento;

// ambos imports compilan sin problema:
// son nombres públicos, no importa el
// paquete en el que estén
```

Con module-info.java (module path)

- Solo se pueden importar tipos de paquetes que el módulo de origen **exportó explícitamente (exports)**.
- Se necesita declarar **requires** del módulo que contiene lo que necesitas importar: si no, **ni siquiera compila**.
- Un tipo **public** en un paquete no exportado **NO se puede importar** desde otro módulo.

```
// módulo biblioteca.pdf declara:
module biblioteca.pdf {
    exports biblioteca.pdf.api;
    // biblioteca.pdf.interno NO se exporta
}

import biblioteca.pdf.interno.Parser; //ERROR
import biblioteca.pdf.api.Documento;  // OK
```

Colisión de nombres

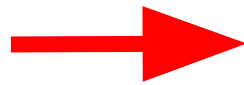
La sentencia **import** puede generar conflictos.

Consideremos los paquetes **java.util** y **java.awt**.

Ambos paquetes contienen el tipo **List**: **java.util.List** es una interface comúnmente usada y **java.awt.List** es una clase.

Supongamos que el paquete pruebas pertenece a un módulo que declara requires java.desktop (necesario para usar java.awt.List).

```
package pruebas;  
import java.util.List;  
import java.awt.List;  
public class ConflictoDeNombres {  
    //TODO  
}
```



¿Se puede hacer?

```
package pruebas;  
import java.util.*;  
import java.awt.*;  
public class ConflictoDeNombres {  
    //TODO  
}
```



¿Se puede hacer?

```
package pruebas;  
import java.util.*;  
import java.awt.*;  
public class ConflictoDeNombres {  
    List l;  
}
```



¿A qué List hace referencia?

Colisión de nombres

¿Cómo lo resolvemos?

Si la interface **java.util.List** es más usada que la clase **java.awt.List** se podría combinar importación de un tipo e importación por demanda y así quitamos la ambigüedad cuando hacemos referencia al tipo **List**.

Esto no cambia si el módulo pruebas declara `requires java.desktop`: los módulos no resuelven ambigüedades de nombre simple, solo habilitan el acceso a los paquetes.

```
package pruebas;
import java.util.List;
public class ConflictoDeNombres {
    List l;
    java.awt.List l2;
}
```

```
package pruebas;
import java.util.*;
public class ConflictoDeNombres {
    List<String> l=new ArrayList<String>();
    java.awt.List l2=new java.awt.List(4);
}
```

En estos casos a la clase **List** de **java.awt** la tenemos que usar con el nombre completo **java.awt.List** y a la interface de **java.util** con el nombre simple.

Colisión de nombres con módulos

Ambigüedad de tipos: no cambia

- `java.util.List` y `java.awt.List` son dos tipos distintos con el **mismo nombre simple**, cada uno exportado por su propio módulo (`java.base` y `java.desktop`).
- Los módulos **no resuelven** esta ambigüedad: se sigue necesitando el nombre calificado, exactamente igual que con paquetes sueltos.

```
module pruebas {
    requires java.desktop;
}

package pruebas;
import java.util.List;
import java.awt.*;
public class ConflictoDeNombres {
    List l1; // OK
    java.awt.List l2 new java.awt.List(4); // OK
}
```

Colisión de paquetes: protección nueva

- Dos módulos **no pueden** exportar el **mismo nombre de paquete** a un mismo consumidor (“**split package**”): es error de compilación.
- En el classpath clásico esto **no se detecta**: el classloader toma las clases del primer JAR que encuentra (un bug silencioso y difícil de diagnosticar).

```
module log.adapter.a {
    exports com.utils;
}

module string.utils.b {
    exports com.utils; // mismo paquete!
}

module mi.app {
    requires log.adapter.a;
    requires string.utils.b;
    // ERROR: split package com.utils
}
```

Importar miembros estáticos

Es posible **importar miembros estáticos** de clases e interfaces usando la palabra clave **import static**.

El **import static** tiene 2 formas:

- Importación de un miembro estático
- Importación de miembros estáticos por demanda

Consideremos la siguiente situación: necesitamos imprimir texto en pantalla usando la salida estándar **System.out**. Usamos la **importación de un miembro estático**:

```
package pruebas;  
import static java.lang.System.out;  
public class ImportOUT {  
    public static void main(String[] args) {  
        out.print("hola");  
    }  
}
```

Consideremos la siguiente situación: hacemos uso exhaustivo de funciones trigonométricas y otras operaciones de la clase **Math**. Usamos **importación de miembros estáticos por demanda**:

Nos evitamos escribir **System.out.print()**

Podemos escribir expresiones concisas sin tener que usar **Math** como prefijo de cada método estático de la clase **Math**.

```
package pruebas;  
import static java.lang.Math.*;  
import static java.lang.System.out;  
public class ImportOUTBis {  
    public static void main(String[] args) {  
        out.println(sqrt(abs(sin(90))) );  
        out.println("Valor de PI: "+PI);  
    }  
}
```

Importar miembros estáticos

La importación estática **importa nombres** NO un miembro específico con dicho nombre. JAVA soporta sobrecarga de métodos y también permite que una clase defina atributos con el mismo nombre que un método -> al importar un miembro estático, podríamos importar más de un miembro (métodos y atributos).

Consideremos el siguiente ejemplo:

```
package pruebas;
import static java.util.Arrays.sort;
public class SobrecargaImportEstatico {
    public static void main(String args[]) {
        String varones[]{"Juan", "Pedro", "Luis", "Ernesto"};
        sort(varones);
    }
}
```

Importa el nombre **sort** no uno de los 18 métodos **sort()** definidos en la clase Arrays

El compilador analizando el tipo, cantidad y orden de los argumentos y determina cuál de los métodos **sort()** queremos usar.

Importar miembros estáticos

```
package pruebas;
import java.util.ArrayList;
import static java.util.Arrays.sort;
import static java.util.Collections.sort;
public class SobrecargaImportEstatico {
    public static void main(String args[]) {
        Integer numeros[]={30,2,44,18,3,23};
        String varones={"Juan", "Pedro", "Luis", "Ernesto"};
        ArrayList<String> mujeres=new ArrayList<String>();
        mujeres.add("Elena");
        mujeres.add("Pilar");
        mujeres.add("Juana");
        sort(varones);
        sort(numeros);
        sort(mujeres);
    }
}
```

Es legal importar métodos estáticos con el mismo nombre, pertenecientes a clases distintas, siempre que los métodos tengan diferentes firmas.

*¿Y con módulos? ¿El **import static** necesita algo más que el **requires** normal del módulo?*

Este código **no tiene errores de compilación** porque los métodos **sort()** definidos en la clase **Collections** tienen diferente firma que los definidos en la clase **Arrays**. El compilador determina cuál de los 20 posibles métodos importados deseamos invocar, analizando el tipo, orden y cantidad de argumentos.

Archivos JAR

Es posible **agrupar múltiples paquetes en un único archivo**:

Usando archivos JAR (Java ARchive)

El formato JAR usa el formato ZIP. Los archivos JAR son multiplataforma, son totalmente portables.

En los archivos JAR pueden incluirse además de archivos **.class** recursos como archivos de imágenes y audio, etc.

La distribución estándar del JSE contiene una herramienta que permite crear archivos JAR desde la línea de comando: el utilitario **jar**.

El “intérprete” JAVA se encarga de buscar, descompactar, cargar y ejecutar los **.class** contenidos en el JAR.

Archivos JAR

El uso de archivos **JAR** es la opción recomendada para la **entrega de aplicaciones o de librerías de componentes**.

Una **aplicación empaquetada en un JAR** es un **archivo ejecutable JAVA** que es posible lanzar directamente desde el sistema operativo.

Los archivos **JAR** además de contener todos los paquetes con sus archivos **.class** y los **recursos de la aplicación**, contienen un archivo **MANIFEST.MF** ubicado en **META-INF/MANIFEST.MF**, cuyo propósito es indicar **cómo se usa el archivo JAR**. Las **aplicaciones de escritorio** a diferencia de las librerías de componentes o utilitarias requieren que el archivo MANIFEST.MF contenga una entrada con **el nombre de la clase** que actuará como **punto de entrada de la aplicación**, la que define el método `main()`.

Manifest-Version: 1.0

Created-By:

Main-Class: capitulo4.paquetes.TestOut.class



Archivos JAR Modulares

Un JAR modular es un JAR común que además incluye **module-info.class** en su raíz. Deja de ser un simple conjunto de .class y pasa a declarar su propio nombre, sus **requires** y sus **exports**.

```
laplataautos-reportes.jar
├── module-info.class          ← módulo
├── META-INF/MANIFEST.MF      ← se mantiene, igual que antes
└── ar/com/laplataautos/reportes/*.class
```

- **EI MANIFEST.MF no cambia por ser modular**

Main-Class y demás atributos siguen funcionando igual; module-info.class es información adicional, no un reemplazo.

- **Ejecución: -p / --module-path en vez de -cp**

```
java -p laplataautos-reportes.jar -m ar.com.laplataautos.reportes
```

Especificadores de acceso

JAVA dispone de facilidades para proveer **ocultamiento de información**: el control de acceso define la accesibilidad a clases, interfaces y a sus miembros, estableciendo **qué está disponible** y **qué no** para los programadores/as que utilizan estas entidades.

Los **especificadores de acceso** determinan la accesibilidad a clases, interfaces y sus miembros y proveen **diferentes niveles de ocultamiento**.

El control de acceso lo podemos utilizar, para:

- Clases e interfaces de nivel superior
- Miembros de clases (métodos, atributos) y constructores

Uno de los factores más importantes que distingue un módulo bien diseñado de uno pobremente diseñado es el nivel de ocultamiento de sus datos y de otros detalles de implementación. Un módulo bien diseñado oculta a los restantes módulos del sistema todos los detalles de implementación separando su interface pública de su implementación.

Desacoplamiento

Reusabilidad de Código

Especificadores de acceso en clases

En JAVA los especificadores de acceso en las declaraciones de clases se usan para determinar **qué clases están disponibles para los programadores/as**. Da lo mismo para interfaces, anotaciones, records.

Una clase declarada **public** es parte de la **API que se exporta** y está disponible mediante la cláusula **import**.

```
package gui;  
public class Control{  
    //TODO  
}  
  
import gui.Control;
```

¿Qué pasa si en el paquete **gui** tengo una clase que se usa de soporte para la clase `Control` y para otras clases del paquete `gui`?

La definimos de acceso **package** y de esta manera solamente puede usarse en el paquete `gui`. Es **privada del paquete**. Es razonable que los miembros de una clase de acceso *package* tengan también acceso **package**.

```
package gui;  
class Soporte{ //TODO}
```

La clase **Soporte** es **privada del paquete gui**: sólo puede usarse en el paquete `gui`

Especificadores de acceso en métodos, variables y constructores

Son **reglas de control de acceso** que restringen el uso de las **variables**, **métodos** y **constructores** de una clase. Permiten que el programador/a de la clase determine **qué está disponible** para el programador/a que usa dicha clase y **qué no**.

Los **especificadores de control de acceso en JAVA** son:

public

protected

package (no tiene palabra clave)

private

Más libre

Más restrictivo

El **control de acceso** permite **ocultar la implementación**. Separa la **interface pública** de la **implementación**, favoreciendo hacer cambios que no afectan al código del usuario de la clase.

En JAVA los **especificadores de acceso** se ubican delante de la definición de cada variable, método y constructor de una clase. El especificador solamente controla el acceso a dicha definición.

¿Qué pasa si a un miembro de una clase no le definimos especificador de acceso?

Tiene acceso por defecto, no tiene palabra clave y comúnmente se lo llama **acceso package o friendly o privado del paquete**. Implica que tienen acceso a dicho miembro solamente las clases ubicadas en el mismo paquete que él. Para las clases declaradas en otro paquete, es un miembro privado.

El acceso **package** le da sentido a agrupar clases en un paquete.

Especificadores de acceso en tipos de datos records y sealed

En records

- Los **atributos** son **private final**, no se pueden declarar public ni protected.
- Los **métodos accesoros** generados automáticamente son siempre **public**.
- El **constructor** tiene la **misma visibilidad** que el record.
- Son implícitamente **final** -> no pueden ser superclase ni subclase, no participan en jerarquías de herencia.

```
package ar.com.laplataautos;

public record Vehiculo (String marca, String nroMotor) { }

// marca y nroMotor: private final
// marca() y nroMotor(): acceso public
```

En sealed interfaces / classes

- Un tipo de datos **sealed** (sellado) es una clase o interface que **restringe** qué otras clases pueden heredar o implementar.
- Los tipos listados en **permits** deben ser accesibles desde donde se declara, en general en el mismo paquete.
- Cada subtipo permitido debe declararse **final**, **sealed** o **non-sealed** (no puede quedar la jerarquía abierta sin decidirlo).
- Combina bien con **record**: records + sealed interface reemplazan a la herencia clásica para modelar jerarquías cerradas de datos.

```
package ar.com.laplataautos;

public sealed interface Vehiculo permits Auto, Camion { }

public final class Auto implements Vehiculo { ... }
public record Camion implements Vehiculo { ... }
```

Especificadores de acceso en métodos, variables y constructores

public

El atributo, método o constructor declarado **public** está disponible para **TODOS**.

```
package labo12;  
public class Auto{  
    public String marca;  
    public Auto() {  
        System.out.println("Constructor de Auto");  
    }  
    void arrancar(){  
        System.out.println("arrancar");  
    }  
}
```

¿Qué observan en este código?

```
import labo12.*;  
public class Carrera{  
    public Carrera() {  
        System.out.println("Constructor de Carrera");  
    }  
    public static void main(String[] args){  
        Auto a=new Auto();  
        System.out.println("Marca: "+ a.marca);  
        a.arrancar();  
    }  
}
```

Especificadores de acceso en métodos, variables y constructores

Privado del paquete (package)

Las variables, métodos y constructores declarados **privados del paquete** son accesibles sólo desde clases pertenecientes al mismo paquete donde se declaran.

```
package labo12;  
public class Auto {  
    public String marca;  
    Auto() {  
        System.out.println("Constructor de Auto");  
    }  
    void arrancar(){  
        System.out.println("arrancar");  
    }  
}
```

¿Qué observan en el código?

¿Qué relación encuentran entre el especificador de acceso package y la herencia?

```
import labo12.*;  
public class Carrera{  
    public Carrera() {  
        System.out.println("Constructor de Carrera");  
    }  
    public static void main(String[] args){  
        Auto a=new Auto();  
        System.out.println("Marca: "+ a.marca);  
        a.arrancar();  
    }  
}  
  
.....  
public class Sedan extends Auto {
```

Especificadores de acceso en métodos, variables y constructores

protected

La palabra clave **protected** está relacionada con la herencia:

- Si se define una **subclase** en un **paquete diferente** al de la **superclase**, la subclase solo tiene acceso a los miembros definidos **public**.
- Si se define una **subclase** en el **mismo paquete** que la **superclase**, la subclase tiene acceso a todos los miembros declarados **public** y **package**.
- El autor de la **clase base** podría **determinar** qué miembros pueden ser **accedidos** por las **subclases** pero no por todo el mundo. Esto es **protected**.
- Además el acceso **protected** provee acceso **package**: las clases declaradas en el mismo paquete que el miembro **protected** tienen acceso a dicho miembro.

```
package labo12;  
public class Auto{  
    public Auto() {  
        System.out.println("Constructor de Auto");  
    }  
    void arrancar(){  
        System.out.println("arrancar");  
    }  
}
```

¿Qué observan en el código?

```
import labo12.*;  
public class Sedan extends Auto{  
    public Sedan() {  
        System.out.println("Constructor de Sedan");  
    }  
    public static void main(String[] args){  
        Sedan x=new Sedan();  
        x.arrancar();  
    }  
}
```

Especificadores de acceso en métodos, variables y constructores

protected

Analizaremos el acceso **protected** aplicado a variables.

```
package labo12;
public class Auto{
protected String marca;
protected String nroMotor;
public Auto() {
    System.out.println("Constructor de Auto");
}
protected void arrancar() {
    System.out.println("arrancar");
}
}
```

¿Qué observan en el código?

```
import labo12.*;
public class Sedan extends Auto{
    private String identificador;
    public Sedan() {
        System.out.println("Constructor de Sedan");
    }
    public void setIdentificador(Auto v){
        this.identificador=this.marca+v.nroMotor;
    }
}
```

El acceso protegido privilegia la relación de herencia: las subclases heredan todas las variables y los métodos protegidos de sus superclases, independientemente del paquete donde estén definidas las clases y las subclases. Además el acceso protegido también es acceso privado del paquete.

Un miembro declarado protegido es parte de la API que se exporta: debe mantenerse siempre, es un compromiso público de un detalle de implementación.

Especificadores de acceso en métodos, variables y constructores

private

Las variables, métodos y constructores declarados **private** solamente están accesibles para la clase que los contiene. Están disponibles para usar dentro de los métodos de dicha clase.

```
public class Postre {  
    private Postre() {  
        System.out.println("Constructor de Auto");  
    }  
}
```

```
public class Helado{  
    public static void main(String[] args){  
        Postre p=new Postre();  
    }  
}
```

¿Qué observan en el código? ¿Qué relación encuentran entre el especificador de acceso **private** y la herencia?

Los **métodos privados** funcionan como **utilitarios de la clase**, sólo pueden invocarse desde otros métodos de la clase donde se declaran, **no puedan reemplazarse** a través del mecanismo de herencia y las modificaciones en su código no “rompen” el código que hace uso de la clase.

Las **variables privadas** forman parte de la **implementación de la clase**, son de uso exclusivo de la misma, sólo pueden manipularse directamente en los métodos de la clase y pueden modificarse sin perjudicar el código que hace uso de la clase.

Una buena práctica es declarar **private** todo lo posible.

Y con los constructores privados ¿qué pasa?

Control de acceso y herencia

La especificación de JAVA establece que una subclase hereda de sus superclases todos los atributos y métodos de instancia **accesibles**:

- Si la subclase está definida en el mismo paquete que la superclase, hereda todos los atributos y métodos de instancia **no-privados**.
- Si la subclase está definida en un paquete diferente que su superclase, hereda solamente los atributos y métodos de instancia **protegidos** y **públicos**.
- Los atributos y métodos de instancia **privados nunca son heredados**.

Los **constructores no se heredan**, se encadenan.

Podría ser confuso: "una subclase NO HEREDA los atributos y métodos de su superclase inaccesibles para ella" -> NO IMPLICA que cuando se crea una instancia de la subclase, no se use memoria para los atributos privados o inaccesibles definidos en la superclase.

Todas las instancias de una subclase incluyen una instancia COMPLETA de la superclase, incluyendo los miembros inaccesibles.

Como los miembros inaccesibles no pueden usarse en la subclase, decimos NO SE HEREDAN.